AWS EC2 Direct Root Login via SSH

AWS Academy Sandbox • Ubuntu • MobaXterm • Root Password Authentication

Practical step-by-step documentation

This document walks through the complete practical from start to finish: starting the AWS Academy sandbox, launching an Ubuntu EC2 instance, connecting to it with MobaXterm, and configuring **direct root login with an SSH password**. Every step includes a screenshot and a short explanation.

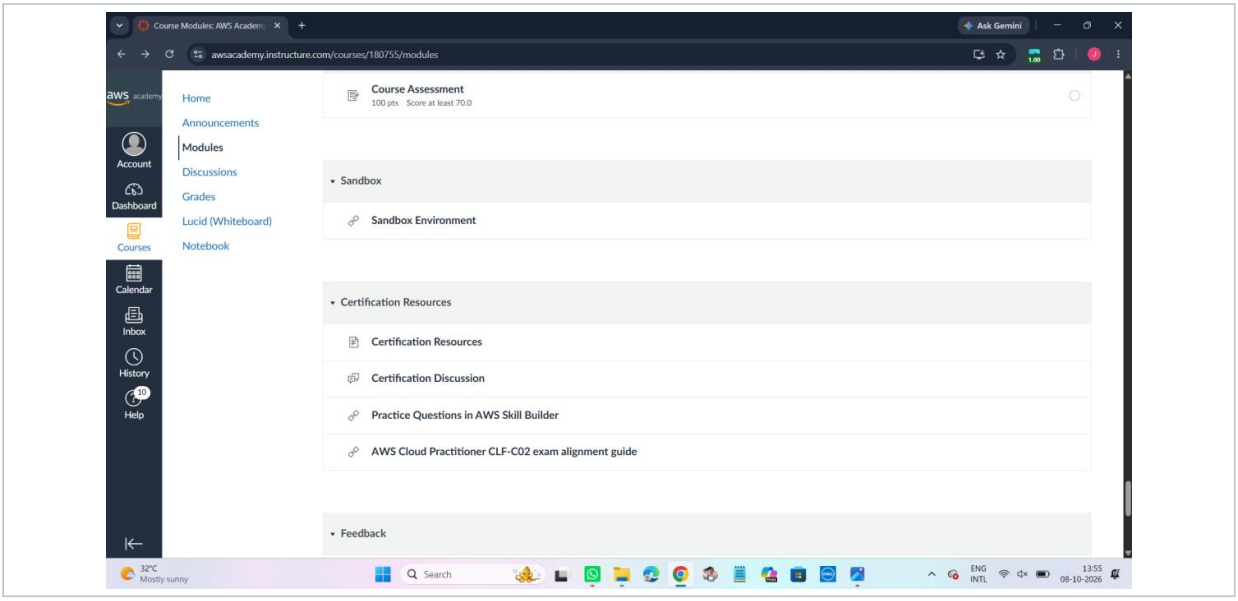
> **Security note:** Direct root login with an SSH password is suitable for a controlled lab demonstration, but it is **not recommended** for a public production server. After the lab, restore the safer SSH settings and use the normal Ubuntu user with sudo.

Part 1 — Start the AWS Academy Sandbox

Open the lab environment that provides a temporary AWS account.

Step 1 — Open the Sandbox Environment module

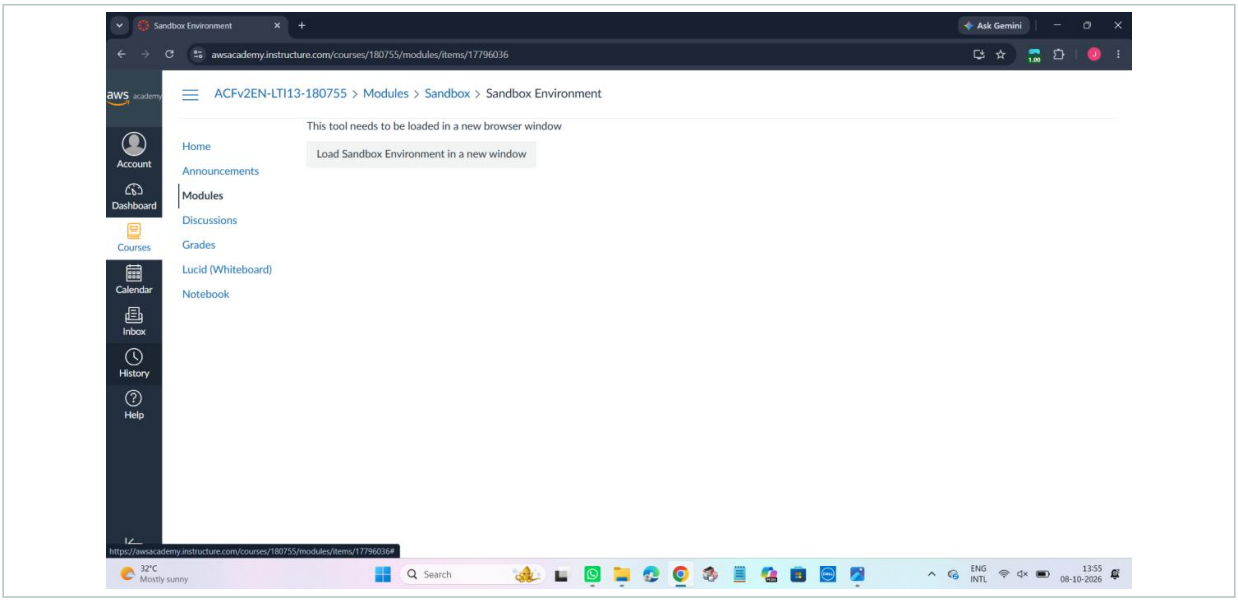
In the AWS Academy course, open **Modules** and scroll down to the **Sandbox** section. Click **Sandbox Environment** to open the lab launcher.



> **What this step demonstrates:** Locate the Sandbox Environment link inside the AWS Academy course modules.

Step 2 — Load the sandbox in a new window

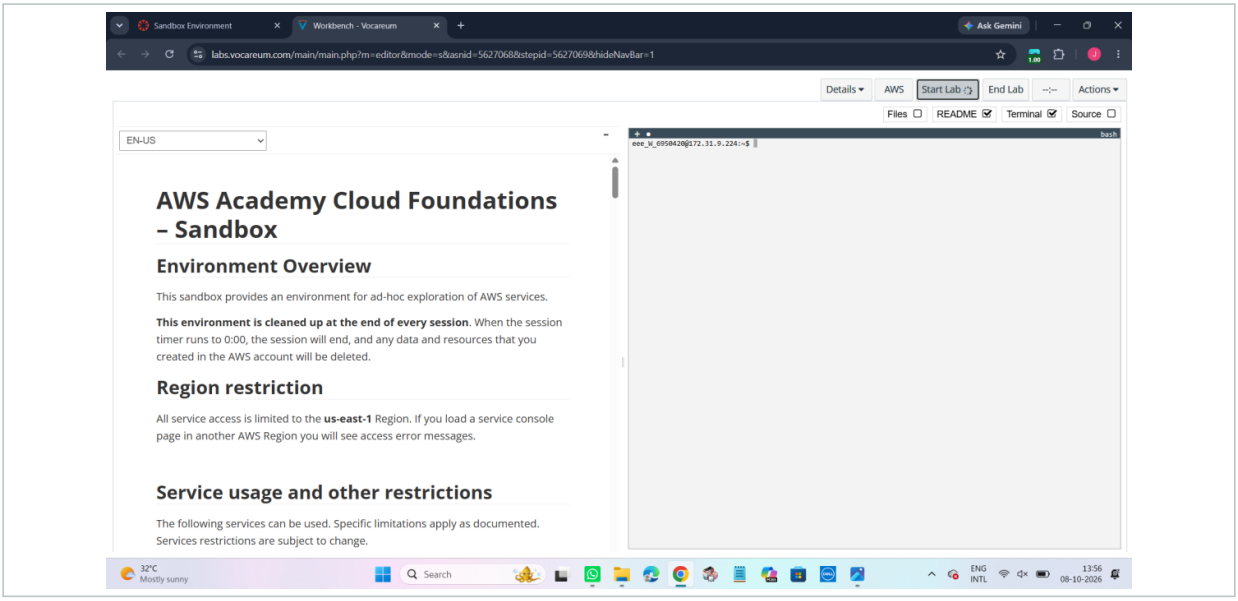
The page states that the tool must be loaded in a new browser window. Click **Load Sandbox Environment in a new window**.



What this step demonstrates: Open the Vocareum workbench, which controls the lab session.

Step 3 — Open the Vocareum workbench and click Start Lab

The workbench shows the AWS Academy Cloud Foundations – Sandbox instructions on the left and a bash terminal on the right. Click **Start Lab** in the top toolbar to create the temporary AWS environment.

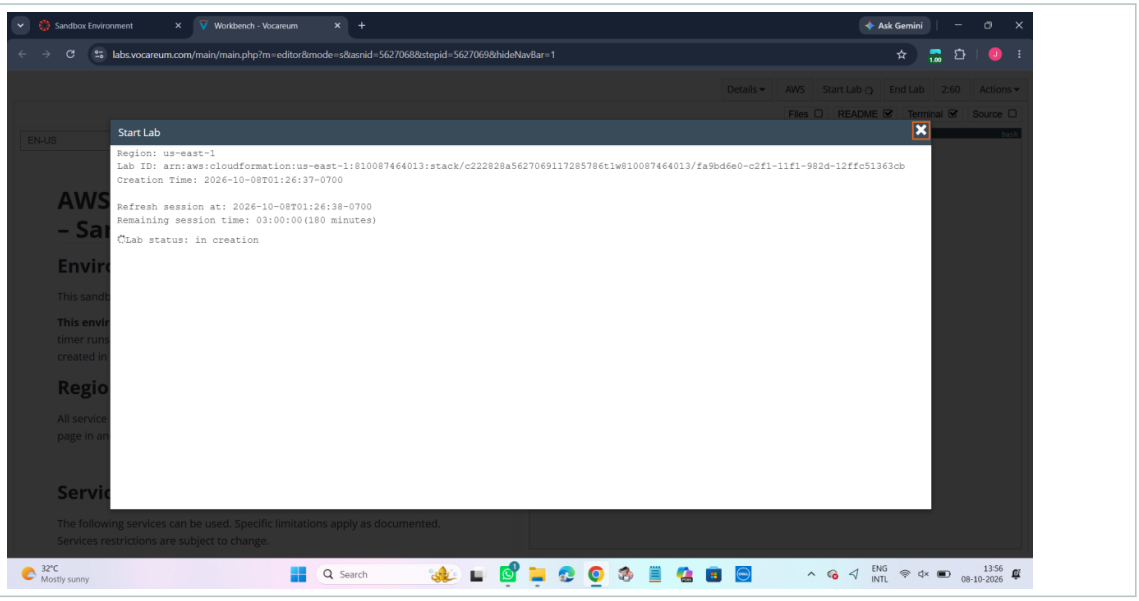


What this step demonstrates: Start the lab session. The README confirms the environment is cleaned up at the end of every session and that all service access is limited to the us-east-1 region.

Tip: All data and resources created in the sandbox are deleted when the session timer reaches 0:00, so save any notes or screenshots before then.

Step 4 — Wait for the lab to be created

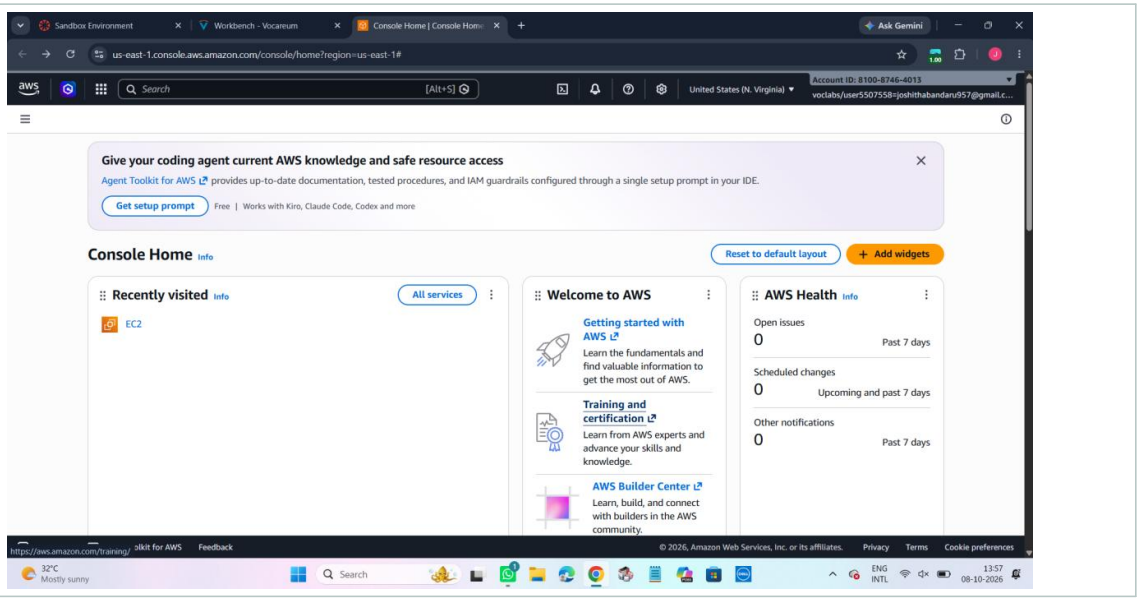
A Start Lab window shows the region (us-east-1), the creation time, and the session length of 180 minutes. The status reads **Lab status: in creation**. Wait until the lab is ready, close the window with the X button, then click **AWS** to open the console.



What this step demonstrates: Confirm the sandbox is being provisioned in us-east-1 with a 3-hour session.

Step 5 — Open the AWS Management Console

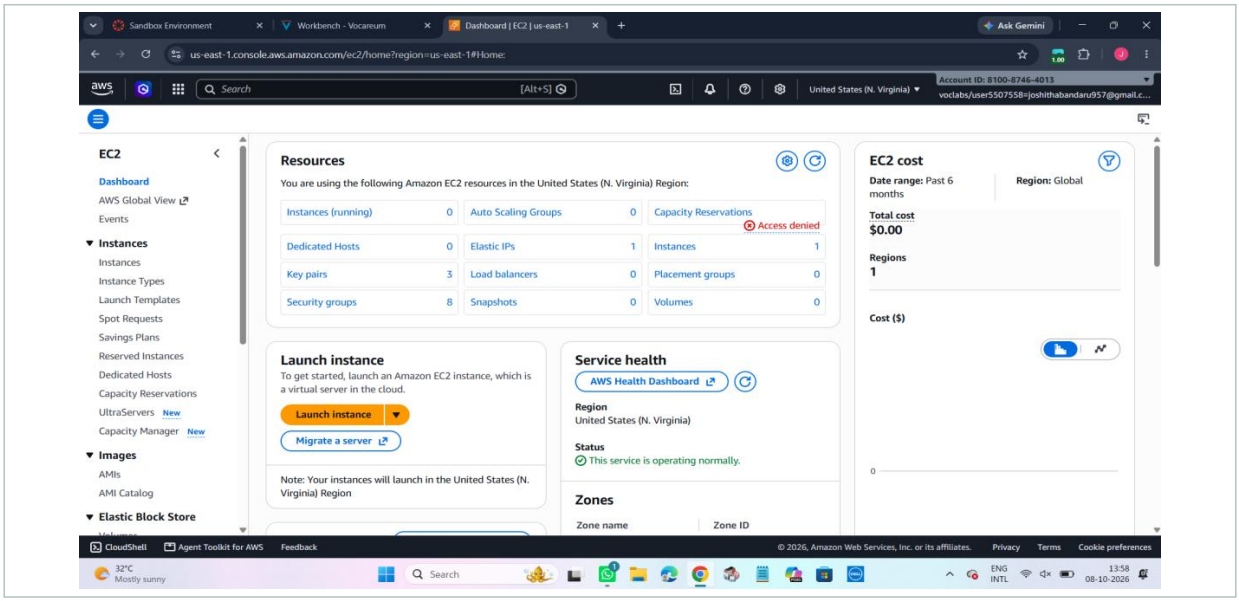
The AWS Console Home opens in United States (N. Virginia). Select **EC2** from the Recently visited list (or search for EC2) to open the EC2 service.



What this step demonstrates: Reach the AWS console and navigate to the EC2 service.

Step 6 — Review the EC2 dashboard

The EC2 dashboard summarises the resources in the region. At this point there are no running instances. Click **Launch instance** to begin creating the server.



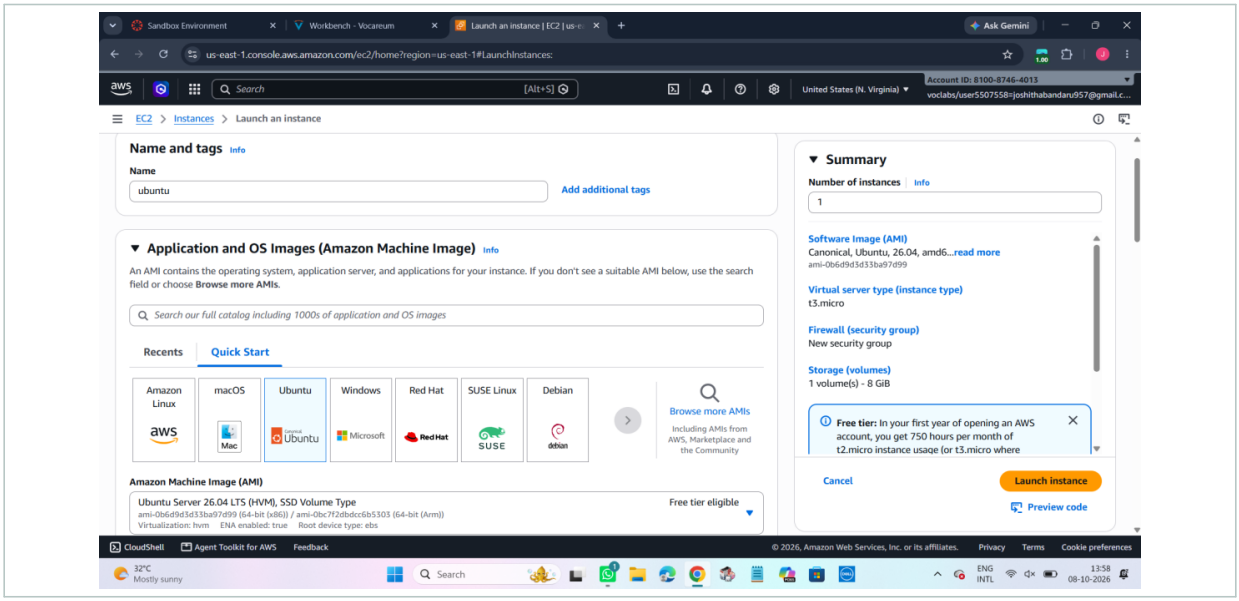
What this step demonstrates: Start the instance-launch wizard from the EC2 dashboard.

Part 2 — Launch the Ubuntu EC2 Instance

Create the server that will be configured for root login.

Step 7 — Name the instance and choose the Ubuntu AMI

Enter the instance name **`ubuntu`**. Under **Quick Start**, select **Ubuntu**. The chosen AMI is **Ubuntu Server 26.04 LTS (HVM), SSD Volume Type**, which is marked Free tier eligible.

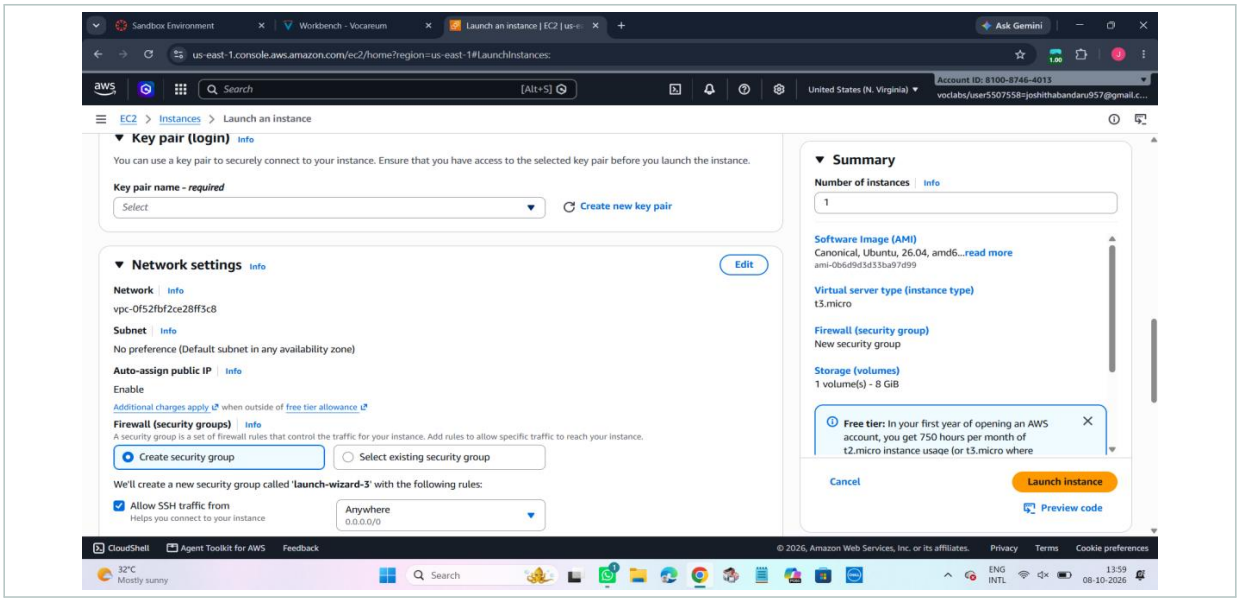


What this step demonstrates: Select the operating system image (Ubuntu Server LTS) for the instance.

Step 8 — Select the key pair and configure network settings

Choose a key pair from the drop-down (or click **Create new key pair** and download the .pem file). This private key is needed for the first login as the ubuntu user.

In **Network settings**, keep the default VPC, leave Auto-assign public IP enabled, and create a new security group (**launch-wizard-3**) with **Allow SSH traffic** turned on. The summary shows instance type **t3.micro** and 8 GiB of storage. Click **Launch instance**.

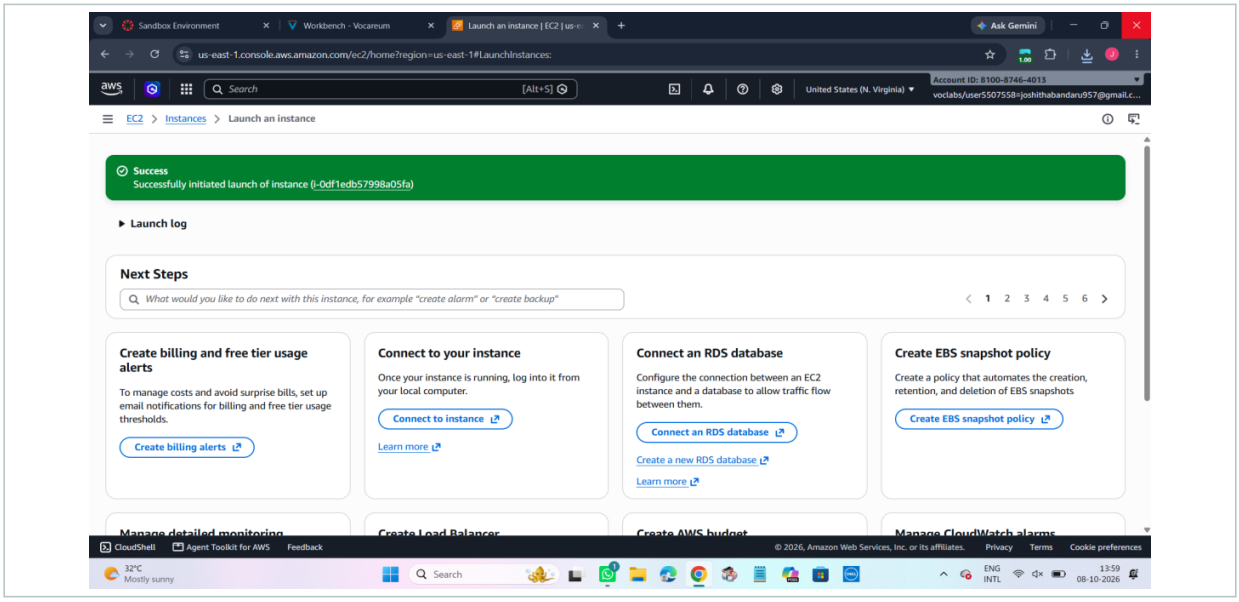


What this step demonstrates: Provide the login key and open SSH (port 22) so the instance can be reached from MobaXterm.

Important: The rule shown allows SSH from **Anywhere (0.0.0.0/0)**. That is acceptable for a short lab, but on a real server restrict SSH to your own IP address.

Step 9 — Confirm the instance was launched

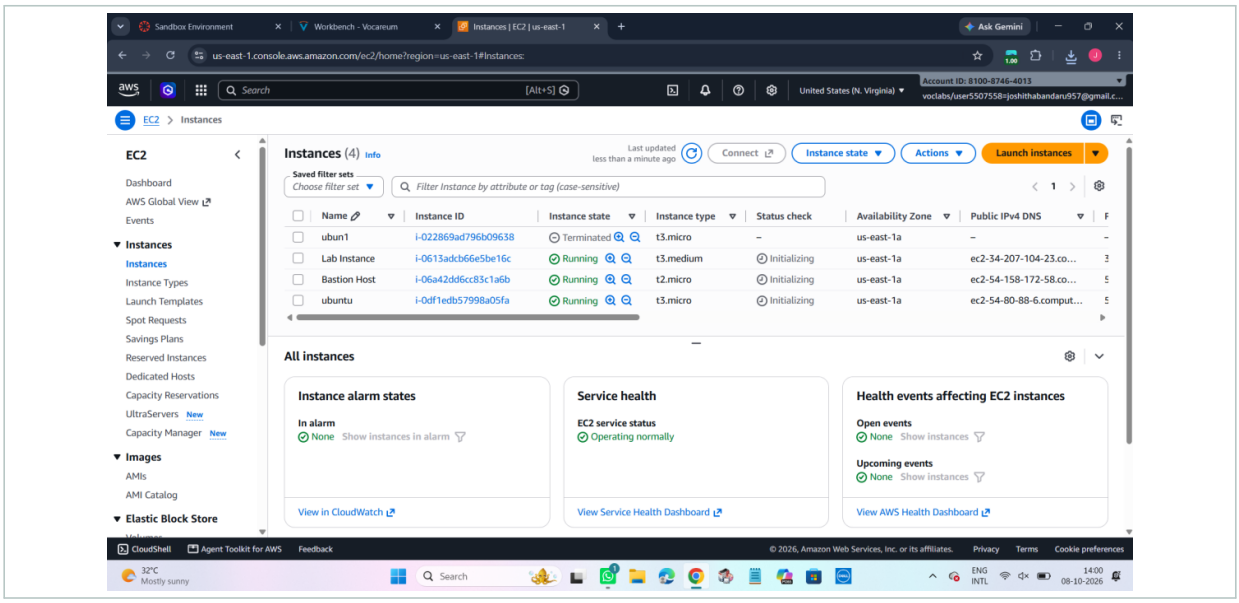
A green **Success** banner confirms the launch and shows the new instance ID, **i-0df1edb57998a05fa**.



What this step demonstrates: Verify that AWS accepted the launch request.

Step 10 — Check the instance list

Open **Instances**. The new **ubuntu** instance (t3.micro) is **Running** and its status check shows **Initializing**. An older instance named *ubun1* is already Terminated and can be ignored.

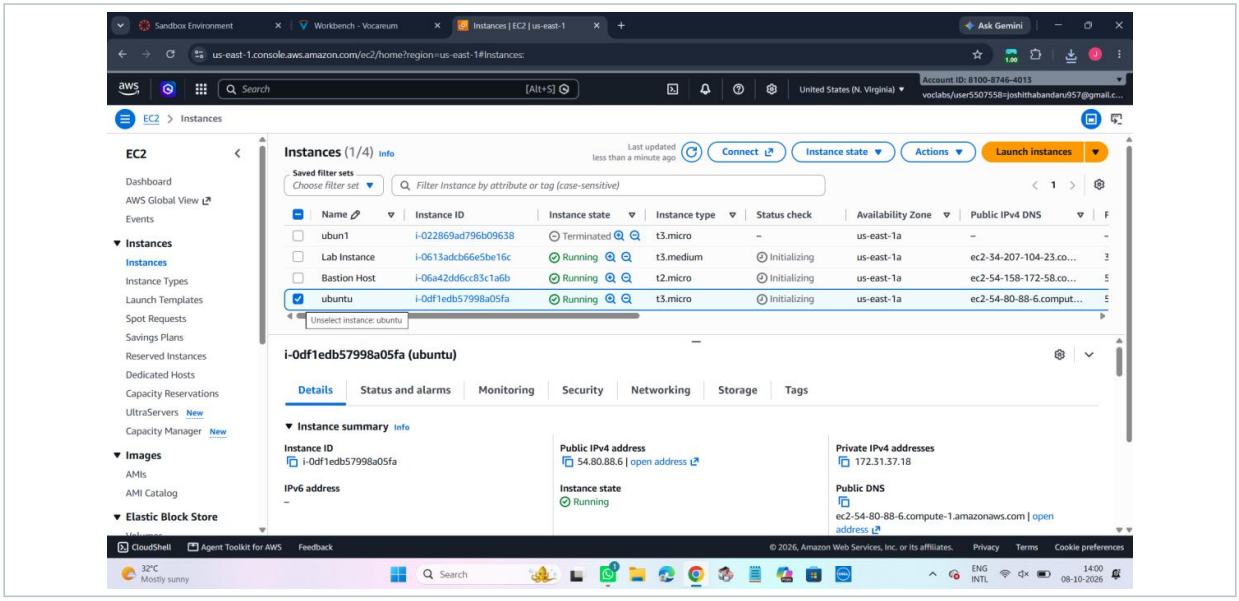


What this step demonstrates: Monitor the instance until it finishes initializing.

Tip: Wait for the status checks to finish before connecting; connecting too early can fail or time out.

Step 11 — Note the public IP address

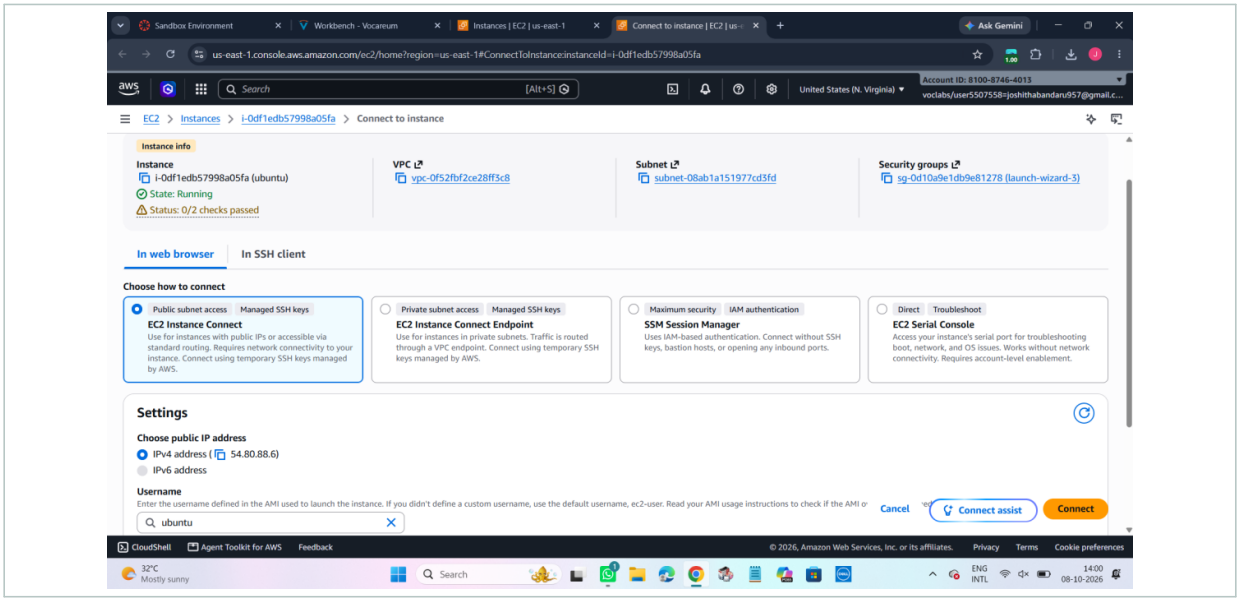
Select the **ubuntu** instance and open the **Details** tab. Note the **Public IPv4 address (54.80.88.6)**, the private address (172.31.37.18) and the public DNS name. The public IPv4 address is used as the host in MobaXterm.



What this step demonstrates: Collect the connection details needed for the SSH session.

Step 12 — Open the Connect to instance page

With the instance selected, click **Connect**. The page lists the instance, VPC, subnet and security group, the public IPv4 address (54.80.88.6) and the default username **ubuntu**.



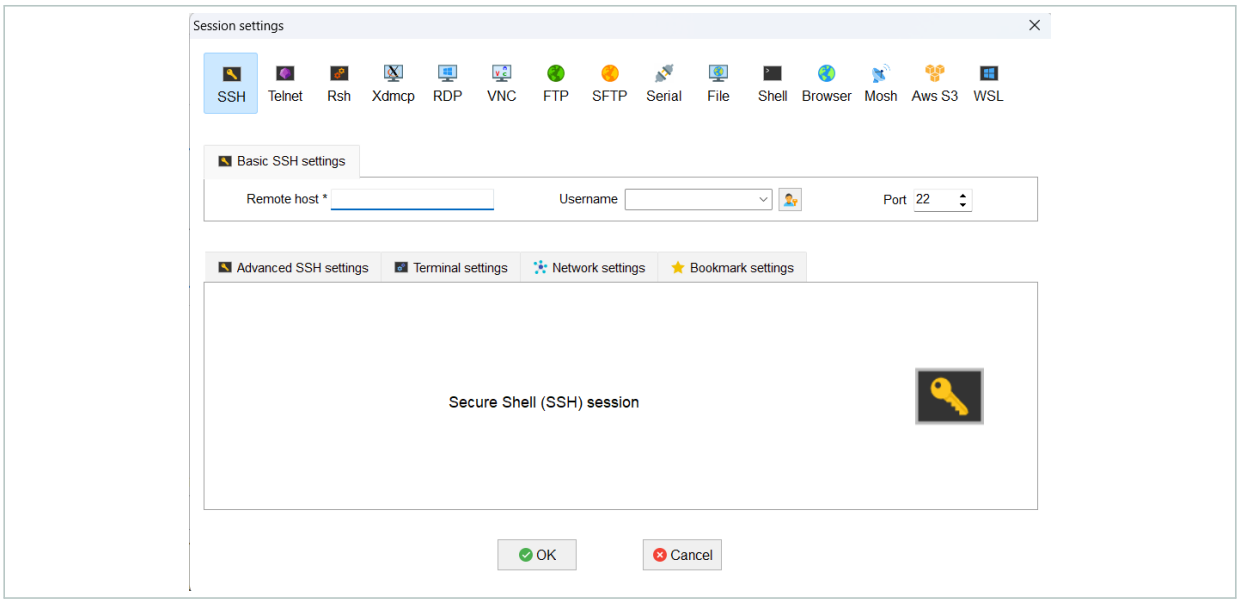
What this step demonstrates: Confirm the default username and public address to use from an external SSH client.

Part 3 — Connect with MobaXterm

Log in as the default ubuntu user using the private key.

Step 13 — Start a new SSH session

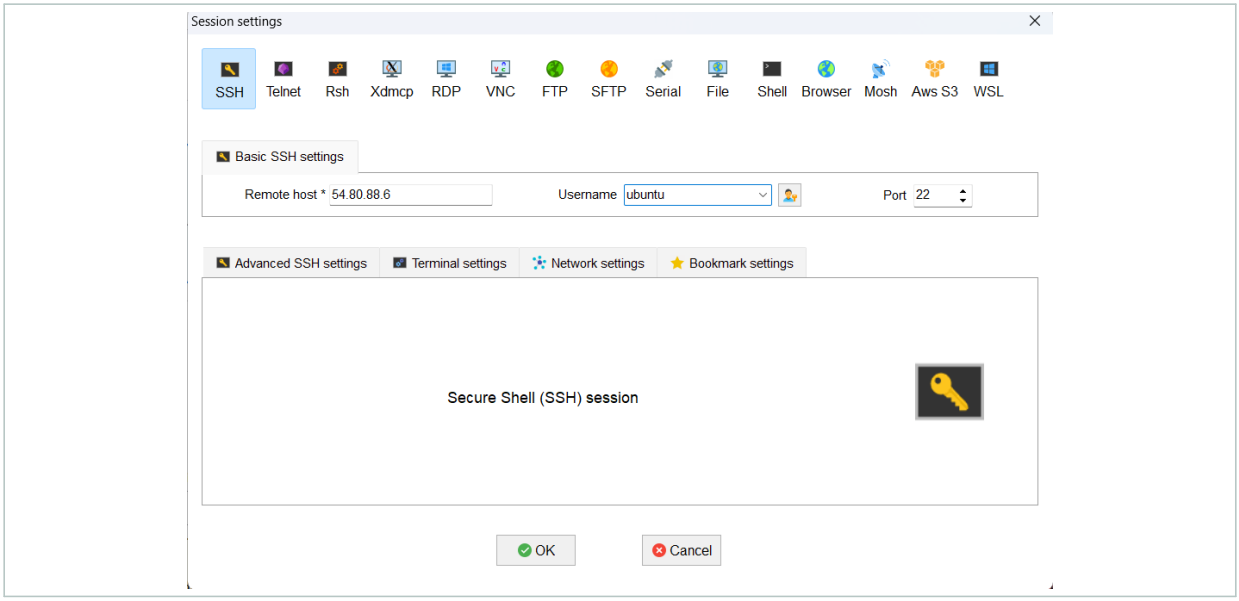
Open MobaXterm and choose **Session** → **SSH**. The Session settings window opens with an empty **Remote host** field.



What this step demonstrates: Begin creating the initial SSH connection to the server.

Step 14 — Enter the host, username and port

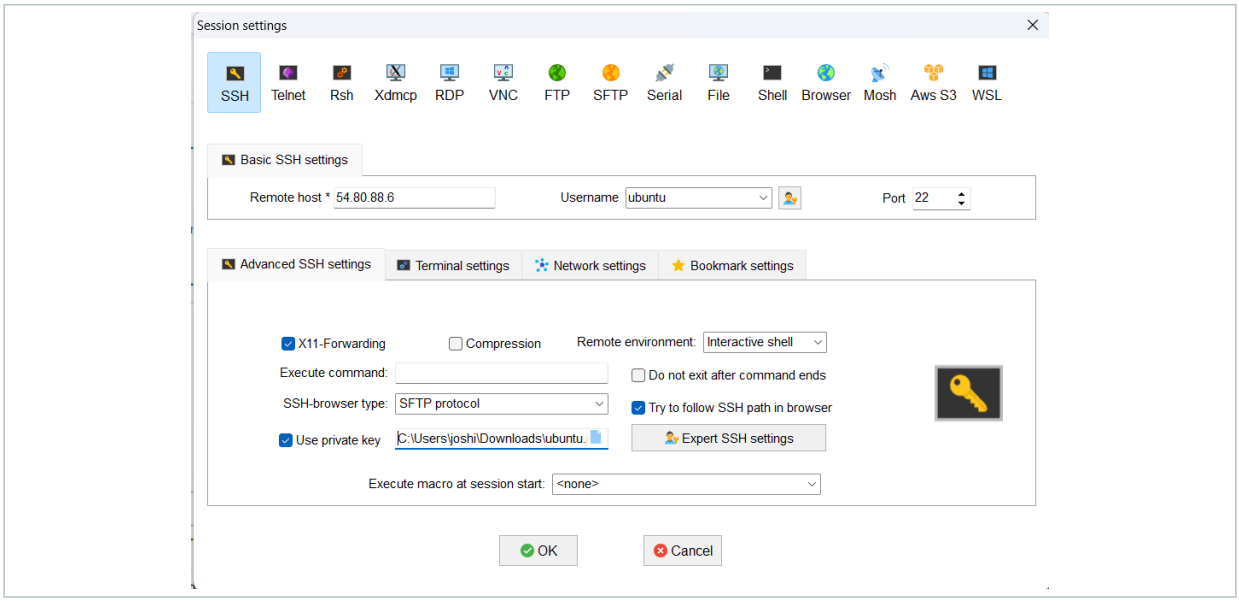
Type the EC2 public IPv4 address **54.80.88.6** in **Remote host**, tick **Specify username** and enter **ubuntu**. Keep the port as **22**.



What this step demonstrates: Define the target server and the initial login user.

Step 15 — Select the private key

Open **Advanced SSH settings**, tick **Use private key**, and browse to the .pem key downloaded when the instance was created (for example in the Downloads folder). Click **OK** to connect.

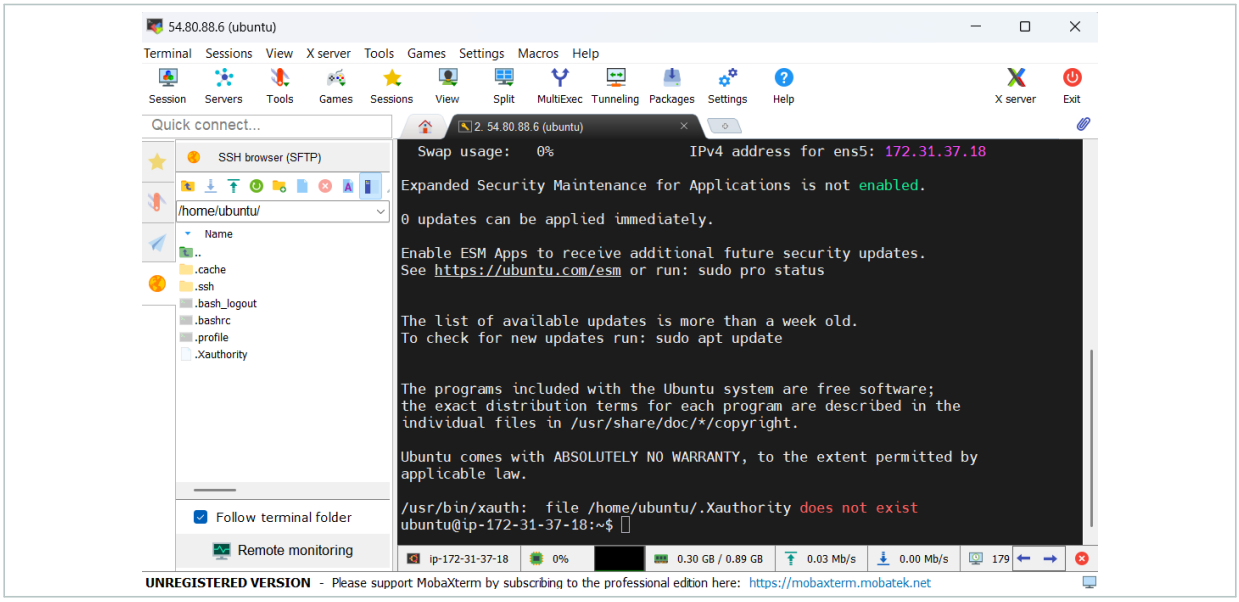


What this step demonstrates: Authenticate the first login with the EC2 key pair.

Important: Never store the private-key file in Git/GitHub or share it.

Step 16 — Verify the Ubuntu SSH login

After connecting, the terminal opens as the Ubuntu user. A prompt similar to **`ubuntu@ip-172-31-37-18:~$`** confirms the SSH connection works. The message that **`/home/ubuntu/.Xauthority`** does not exist is a harmless X11-forwarding notice on first login.



What this step demonstrates: Confirm successful access before changing any SSH configuration.

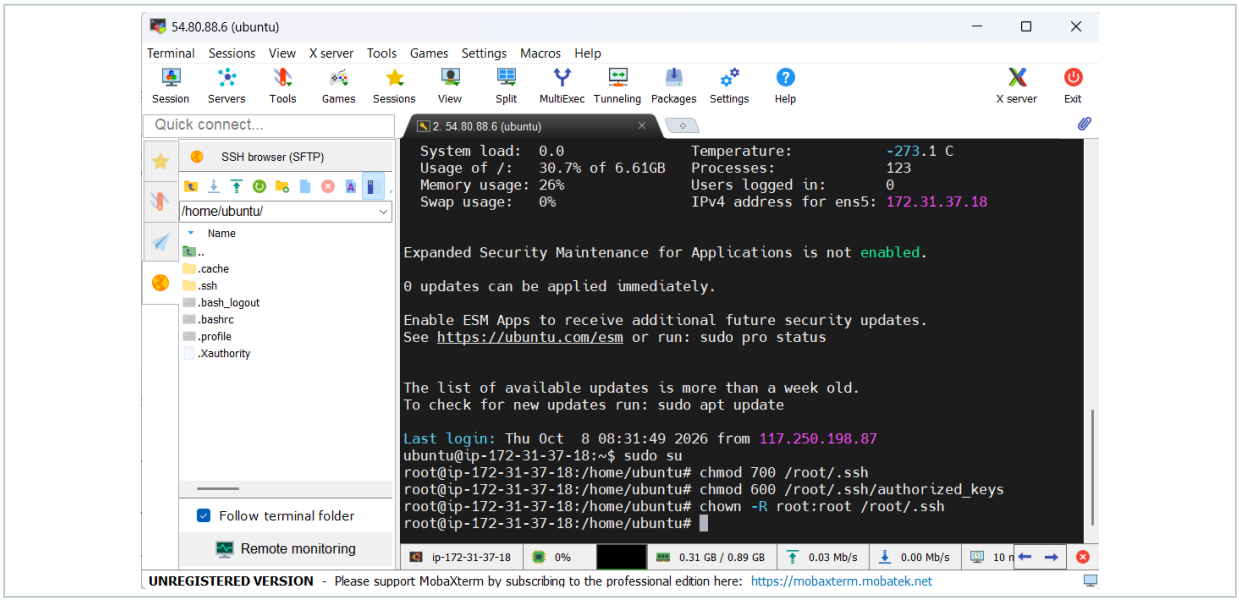
Part 4 — Enable Direct Root Login

Configure the SSH daemon, set a root password and test the root session.

Step 17 — Switch to root and secure the root .ssh folder

Switch to the root shell and set safe permissions and ownership on root's SSH folder:

```
sudo su
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
chown -R root:root /root/.ssh
```



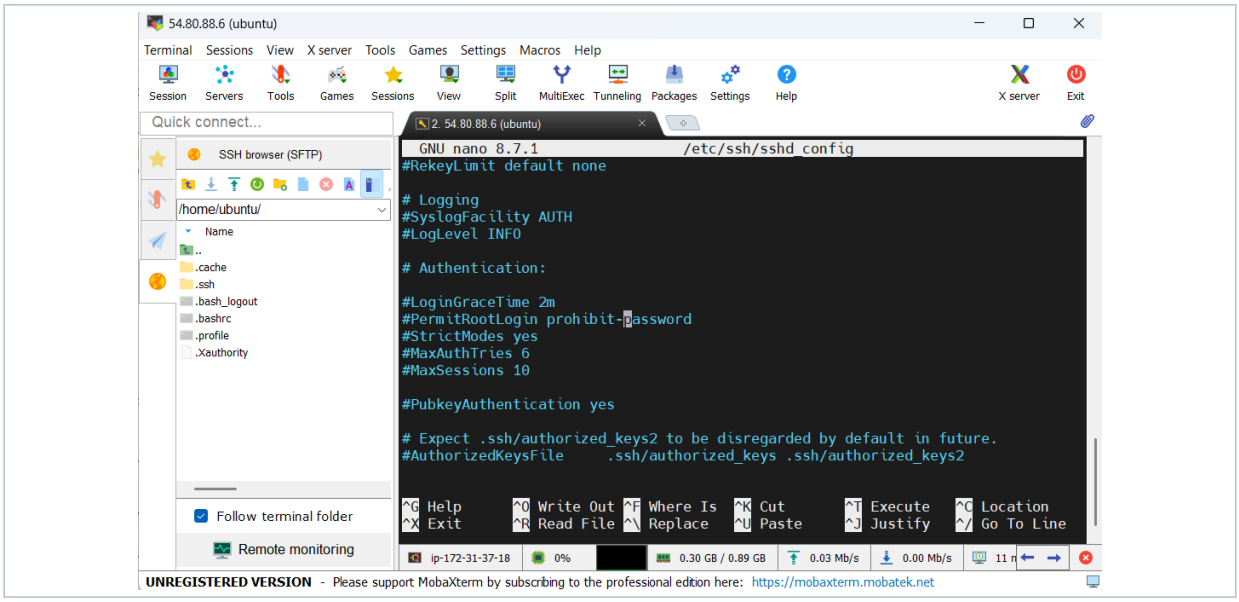
What this step demonstrates: Prepare root's SSH directory with the permissions that the SSH server requires.

Step 18 — Open the SSH server configuration

Open the SSH daemon configuration file in the nano editor and find the **Authentication** section. By default the line **`#PermitRootLogin prohibit-password`** is commented out.

Remove the # and set the options on separate lines: **`PermitRootLogin yes`** and **`PasswordAuthentication yes`**. Save with **Ctrl+O**, press **Enter**, and exit with **Ctrl+X**.

```
nano /etc/ssh/sshd_config
```



What this step demonstrates: Allow the SSH daemon to accept direct root login with password authentication.

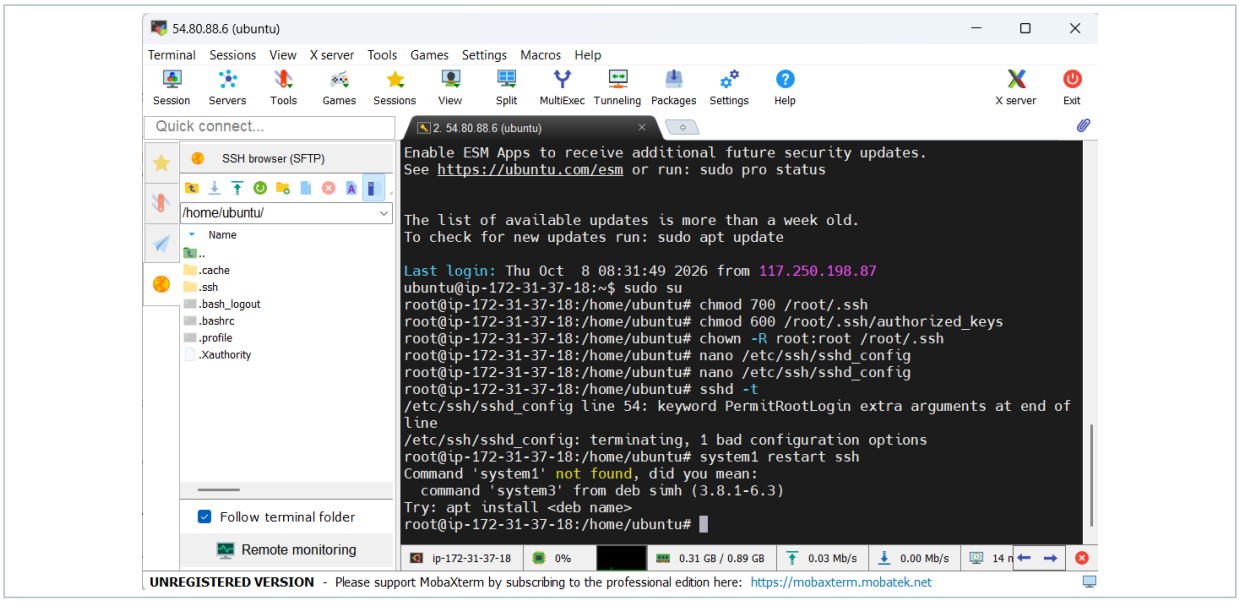
Important: For a public AWS server, direct root/password SSH access is less secure and should normally be disabled after the lab.

Step 19 — Validate the configuration and read the error

Check the syntax with **`sshd -t`**. In this run it reported **line 54: keyword PermitRootLogin extra arguments at end of line** and **terminating, 1 bad configuration options**. This usually means a second setting was typed on the same line as PermitRootLogin.

The attempt to restart the service also failed because of a typing mistake (**`system1`**); the correct command is **`systemctl`**.

```
sshd -t
```



What this step demonstrates: Catch configuration mistakes before restarting SSH.

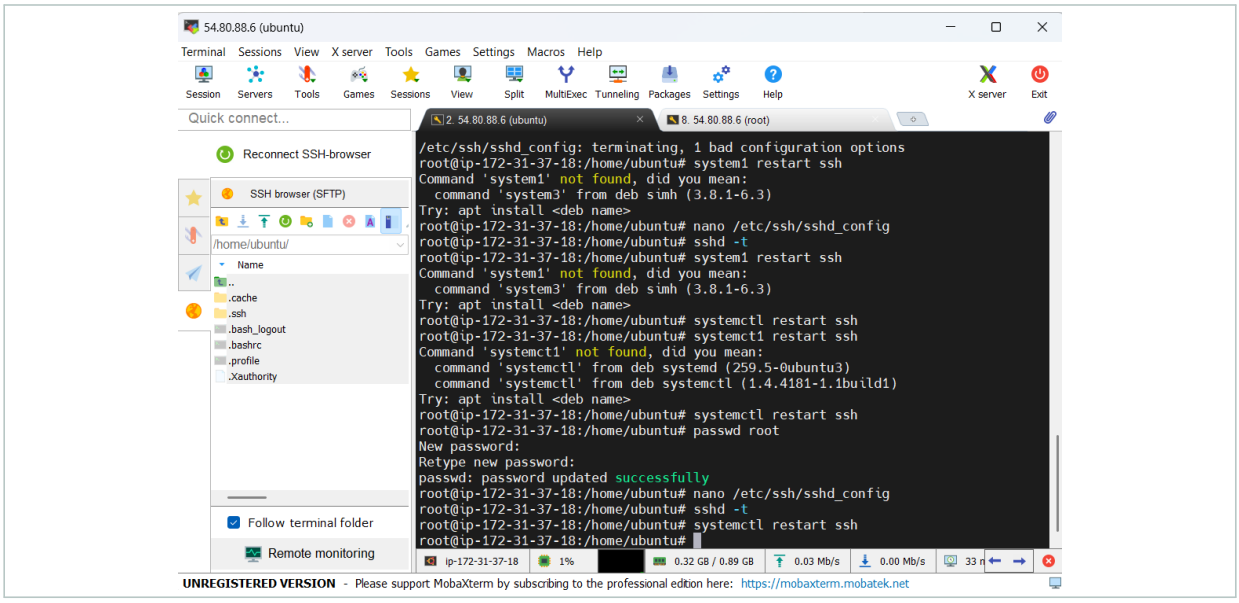
Tip: Keep this working Ubuntu session open until the root login has been tested, so you can still fix the file if something goes wrong.

Step 20 — Fix the file, set the root password and restart SSH

Re-open the file, correct the line so each option is on its own line, and run **`sshd -t`** again. No output means the syntax check passed. Restart the service with **`systemctl restart ssh`** (earlier typos such as **`systemct1`** return “command not found”).

Set the root password with **`passwd root`**. The terminal confirms **password updated successfully**. The validation and restart were then run once more to apply the final settings.

```
nano /etc/ssh/sshd_config
sshd -t
systemctl restart ssh
passwd root
```



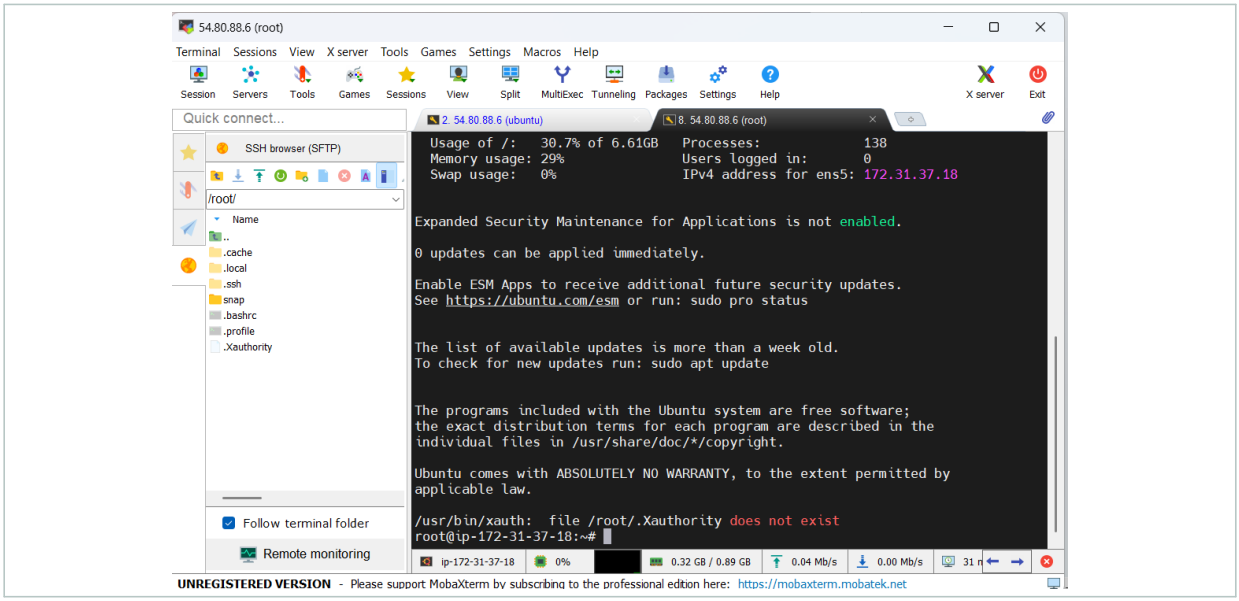
What this step demonstrates: Create the root password and apply the SSH configuration safely.

Important: Never store the actual root password in Git/GitHub or in this document.

Step 21 — Log in directly as root

Create a new MobaXterm SSH session using the same host (**54.80.88.6**), username **root** and port **22**. Leave **Use private key** unchecked so MobaXterm asks for the root password, then enter the password created with passwd root.

A second tab, **54.80.88.6 (root)**, opens and the prompt now ends in # (**`root@ip-172-31-37-18:~#`**).



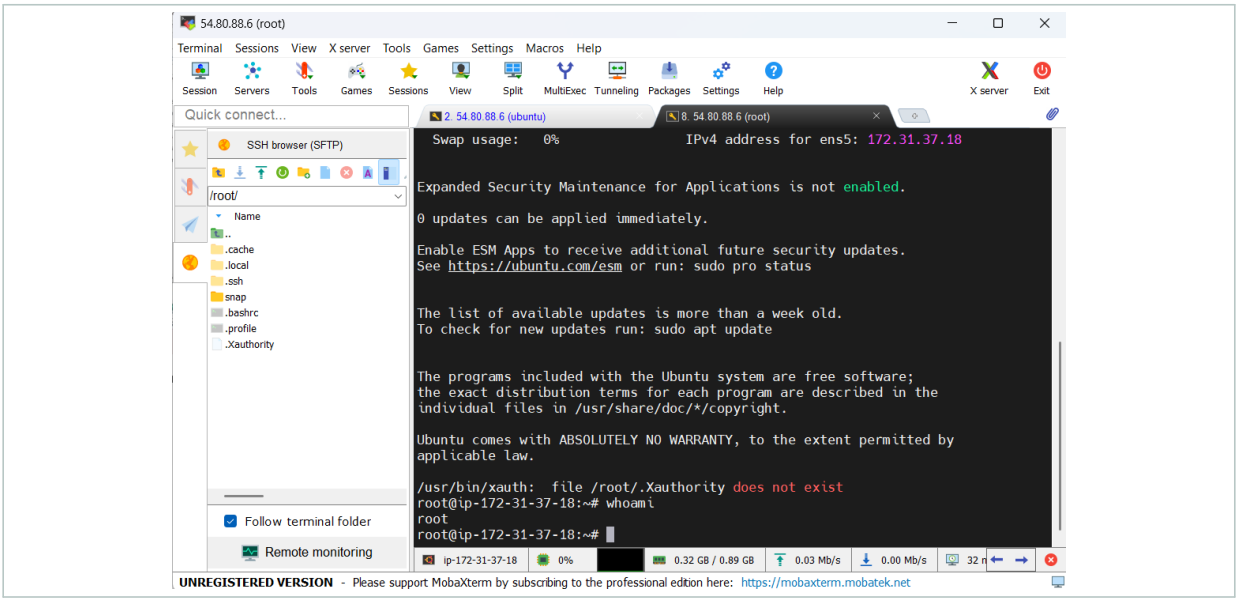
What this step demonstrates: Authenticate directly as root with a password instead of using sudo after logging in as ubuntu.

Tip: Expected result: the session opens as root and shows a prompt ending in #.

Step 22 — Final verification with whoami

Run whoami in the root tab. The output is **root**, confirming the login identity.

```
whoami
```



What this step demonstrates: Result: direct root + password SSH login completed. The server is reachable as root without first logging in as ubuntu and running sudo su.

After the Lab

Restore safe settings and review the commands used.

Once the demonstration is finished, edit **`/etc/ssh/sshd_config`** again and set **`PermitRootLogin prohibit-password`** and **`PasswordAuthentication no`**, then run **`systemctl restart ssh`**. Use the ubuntu user with sudo for normal administration. Finally, click **End Lab** in the workbench; the sandbox deletes all resources when the session ends.

Commands used

Command	Purpose
`sudo su`	Switch to the root shell
`chmod 700 /root/.ssh`	Restrict the root .ssh folder to its owner
`chmod 600 /root/.ssh/authorized_keys`	Restrict the authorized keys file
`chown -R root:root /root/.ssh`	Make root the owner of the folder
`nano /etc/ssh/sshd_config`	Edit the SSH server settings
`sshd -t`	Check the configuration syntax
`systemctl restart ssh`	Apply the new configuration
`passwd root`	Set the root account password
`whoami`	Confirm the current user is root